

Introduction to Computing (CS 101)

Problem Solving, Computing and Programming

T. Venkatesh & John Jose



Department of Computer Science & Engineering
Indian Institute of Technology Guwahati

How to Solve a Problem using Computer?

- Given a Problem :
 - In English description
- Solve using Computer

Computing

- Design methods to solve the problem
- Analyze the designed solution for correctness
- Design a flow chart to for the design method

- Write an Algorithm/Pseudocode
- Choose a programming language and write the code
- Compile the code : **You may get some error**
- Test the code (with some Input): **you get some error**

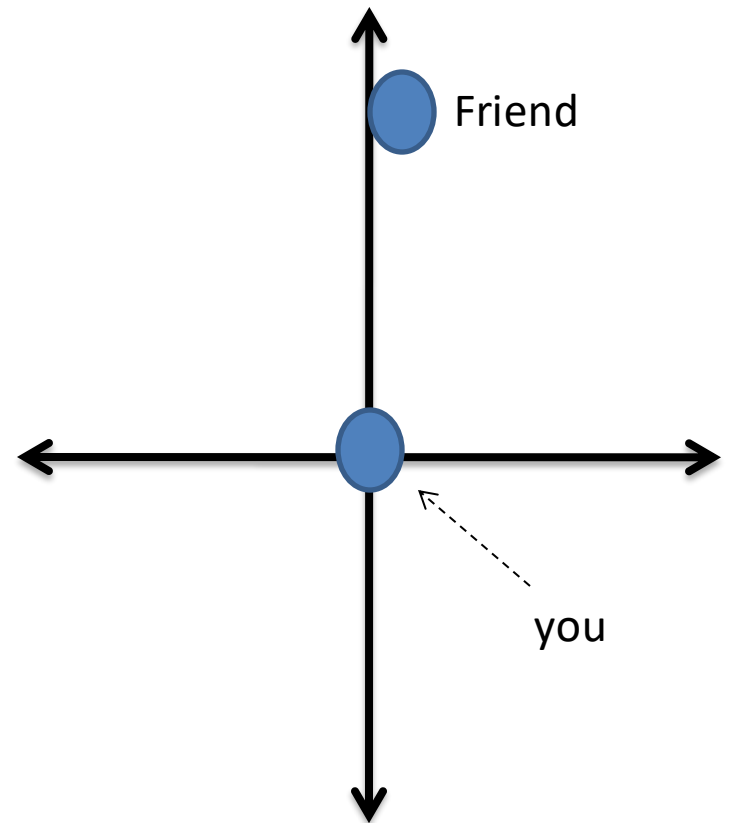
Programming

Problem Solving Example 1

- Given a positive integer, is this number divisible by 3?
- Method 1
 - Divide the number by three and test for the remainder
 - Repeated subtraction of 3 till you get the result less than 3
 - Divide using usual method and get remainder
- Method 2
 - Sum all the digits of the number and test the divisibility of Sum by three, repeat the same.
 - If sum is divisible by three then number is divisible by three. How you got to know?

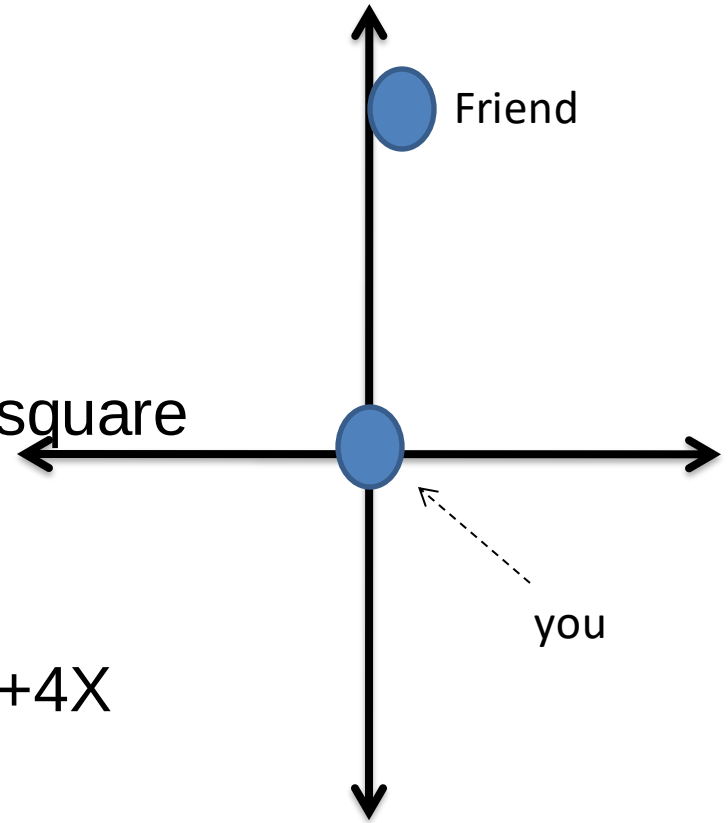
Problem Solving Example 2

- Problem: Searching for your friend
 - Your friend is near to square and you are exactly at the center of the square
 - You don't know which direction and how much distance from the center of the square
 - You need to find your friend
- Any Solution?
 - How you will search him?
 - Any approach ? Think



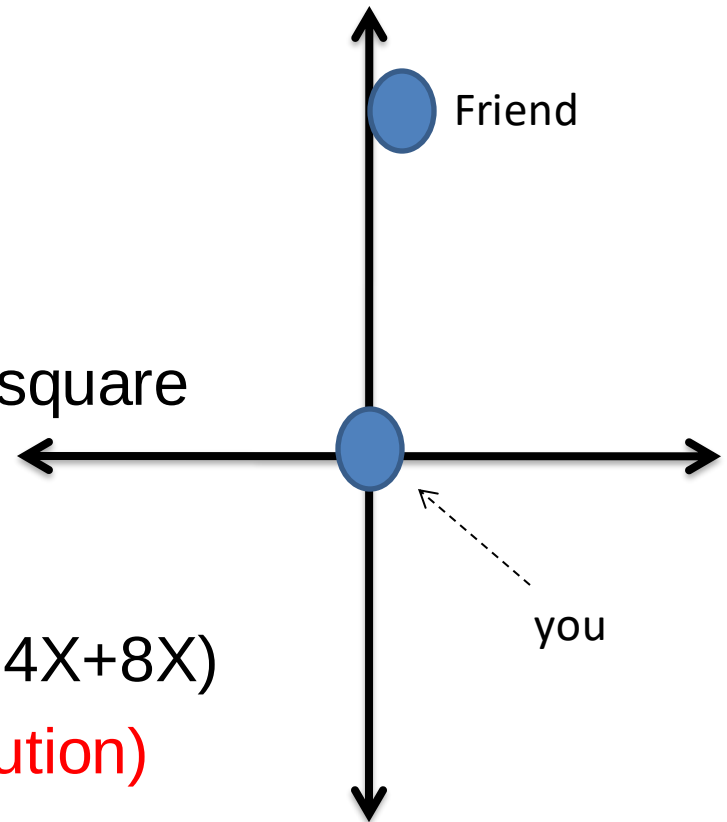
Problem Solving Example 2

- Problem: Searching for your friend
 - You don't know which direction and how much distance from the center of the square
- Solution?
 - $K=1$;
 - While (not found)
 - For all directions
 - Go K meter and return to square
 - $K=K+1$;
- Analysis :Total distance covered
 - $4 \cdot 2(1+2+\dots+X) = 8 \cdot X \cdot (X+1)/2 = 4X^2 + 4X$



Problem Solving Example 2

- Problem: Searching for your friend
 - You don't know which direction and how much distance from the center of the square
- Better Solution? **YES**
 - $K=1$;
 - While (not found)
 - Go in Next directions
 - Go K meter and return to square
 - $K=K*2$;
- Analysis: distance covered
 - At max $2*(1+2+2^2+2^3+..+X+2X+4X+8X)$
 $< 31*X < 4X^2+4X$ (earlier solution)



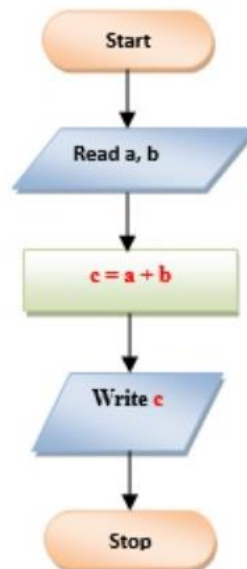
Algorithm and Flowchart

- **Algorithm:** Step by step process or set of rules to be followed in calculations or other problem-solving operations, to be done on a computer.
- **Flowchart:** It is a graphical representation of the steps in an algorithm
- Emphasizes individual steps

Algorithm

1. Start
2. Read a, b
3. $c = a + b$
4. Print or display c
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int a, b, c;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);
    c = a+b;

    printf("Sum of given two numbers is: %d", c);

    return 0;
}
```

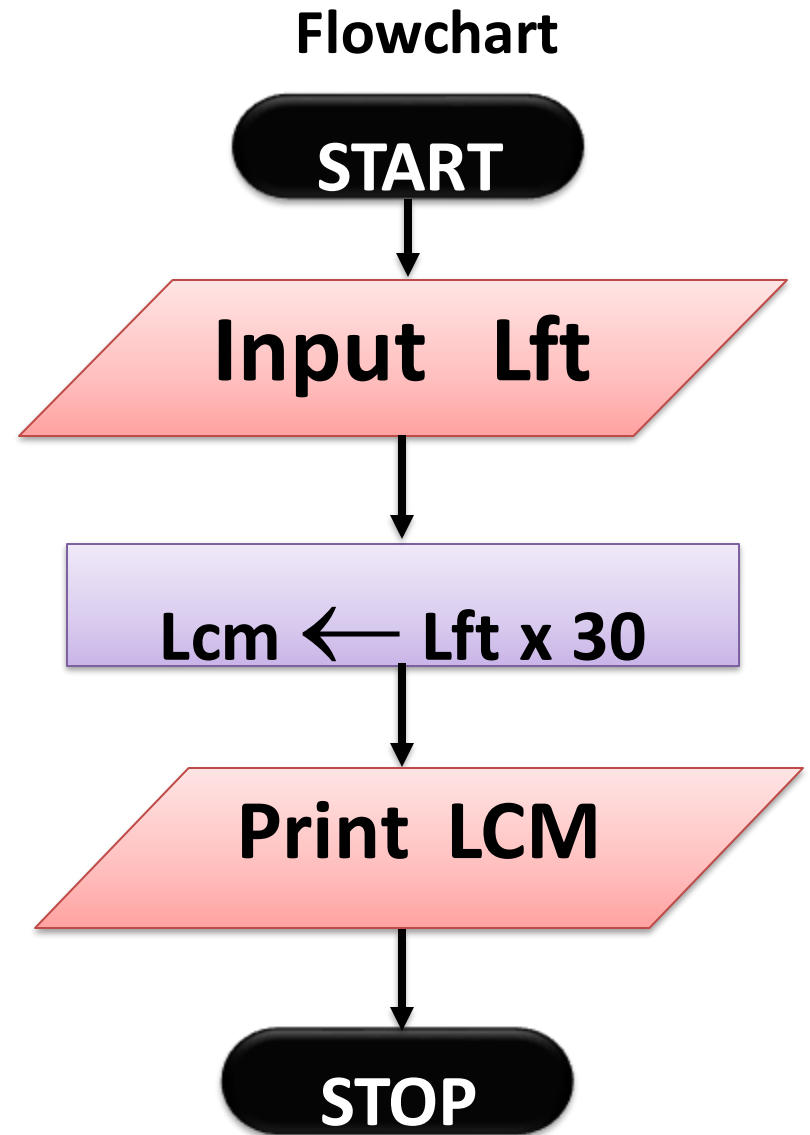
Flowchart Symbols



- **Oval:** begin and end
- **Parallelogram:** Input or Output
- **Rectangle:** process to be carried out
 - add/sub/div/inc
- **Diamond:** Decision/Branch to be made IF/THEN/ELSE
- **Flow line:** Direction of logic flow in program

Example 1: Length FT to Length CM

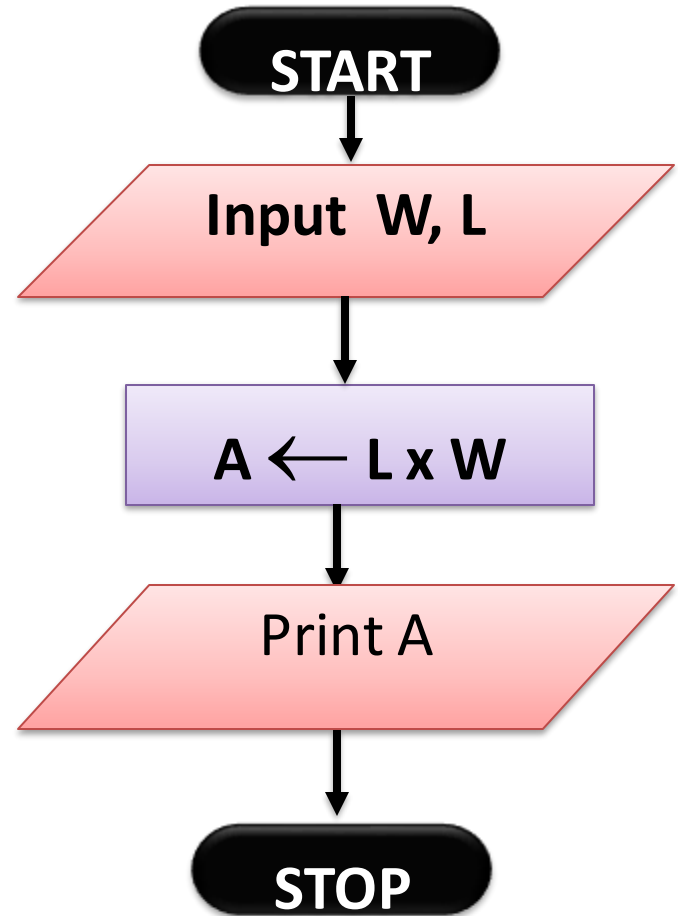
- Step 1: Input Lft
- Step 2: $L_{cm} \leftarrow L_{ft} \times 30$
- Step 3: Print Lcm



Example 2: Area of Rectangle

Read the two sides of a rectangle and calculate its area.

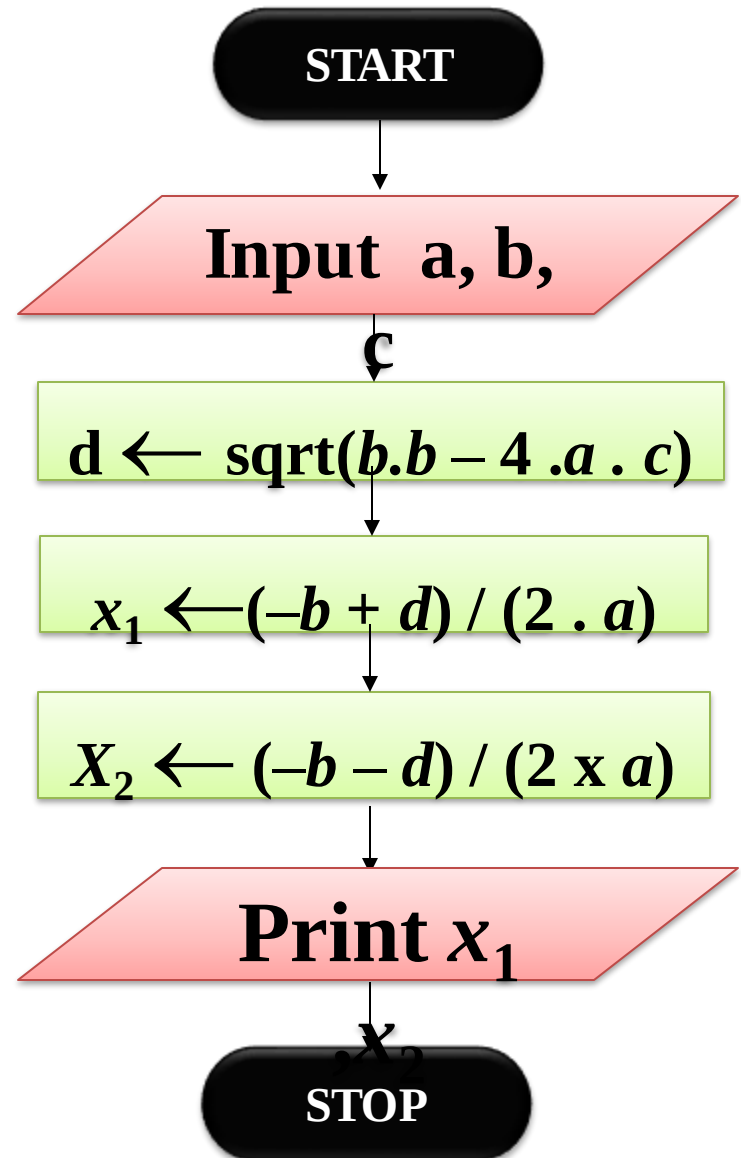
- Step 1: Input W,L
- Step 2: $A \leftarrow L \times W$
- Step 3: Print A



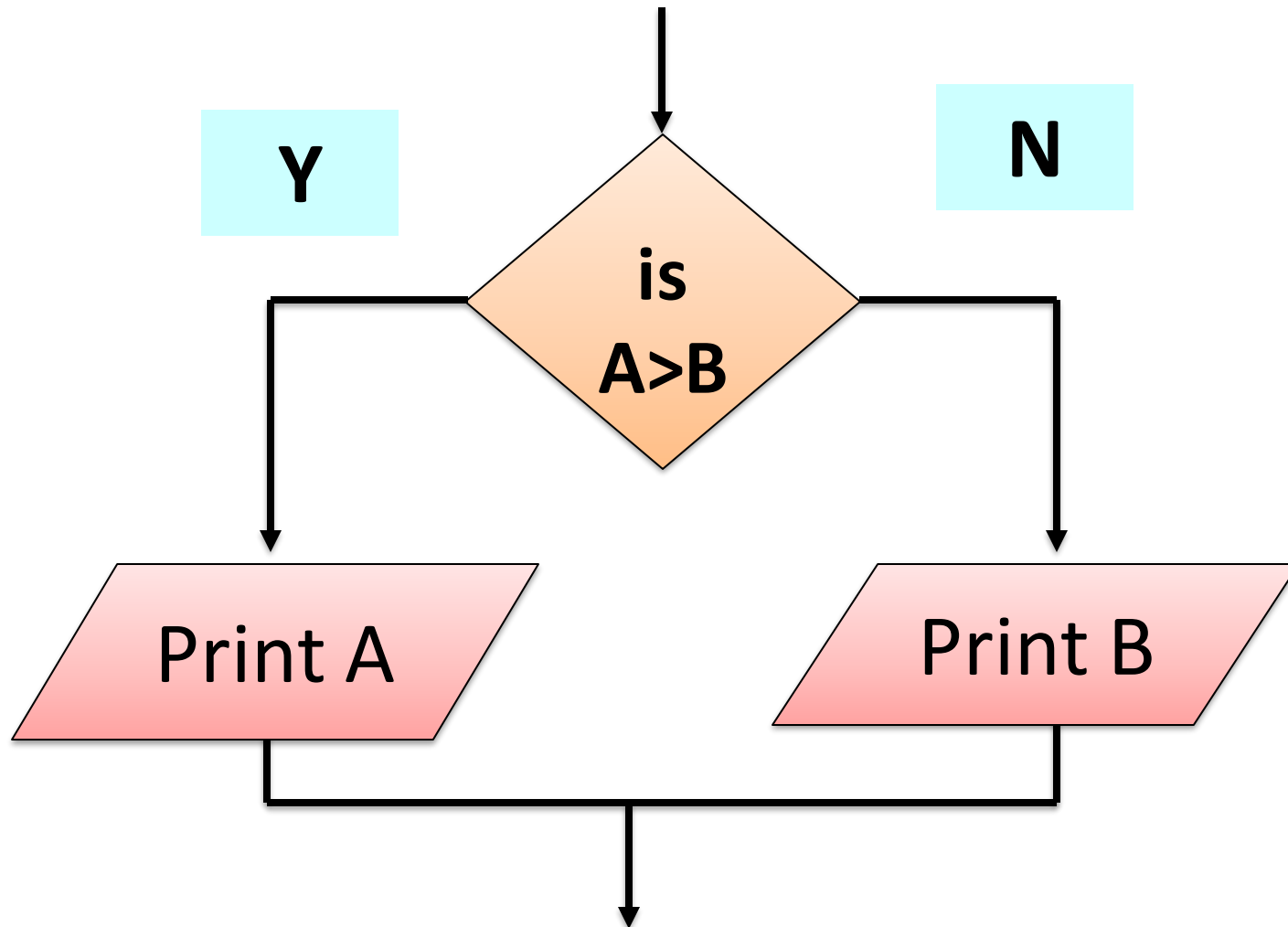
Example 3 : Roots of a Quadratic Equation

- Quadratic equation $ax^2+bx+c=0$
- Calculate $d = \text{sqrt} (b^2-4*a*c)$
- Roots are: $x_1 = (-b + d)/2a$
 $x_2 = (-b - d)/2a$

- Step 1: Input a, b, c
- Step 2: calculate d
 $d \leftarrow \text{sqrt} (b^2-4*a*c)$
- Step 3: calculate x1
 $x_1 \leftarrow (-b + d) / (2 * a)$
- Step 4: calculate x2
 $x_2 \leftarrow (-b - d) / (2 * a)$
- Step 5: Print x1, x2

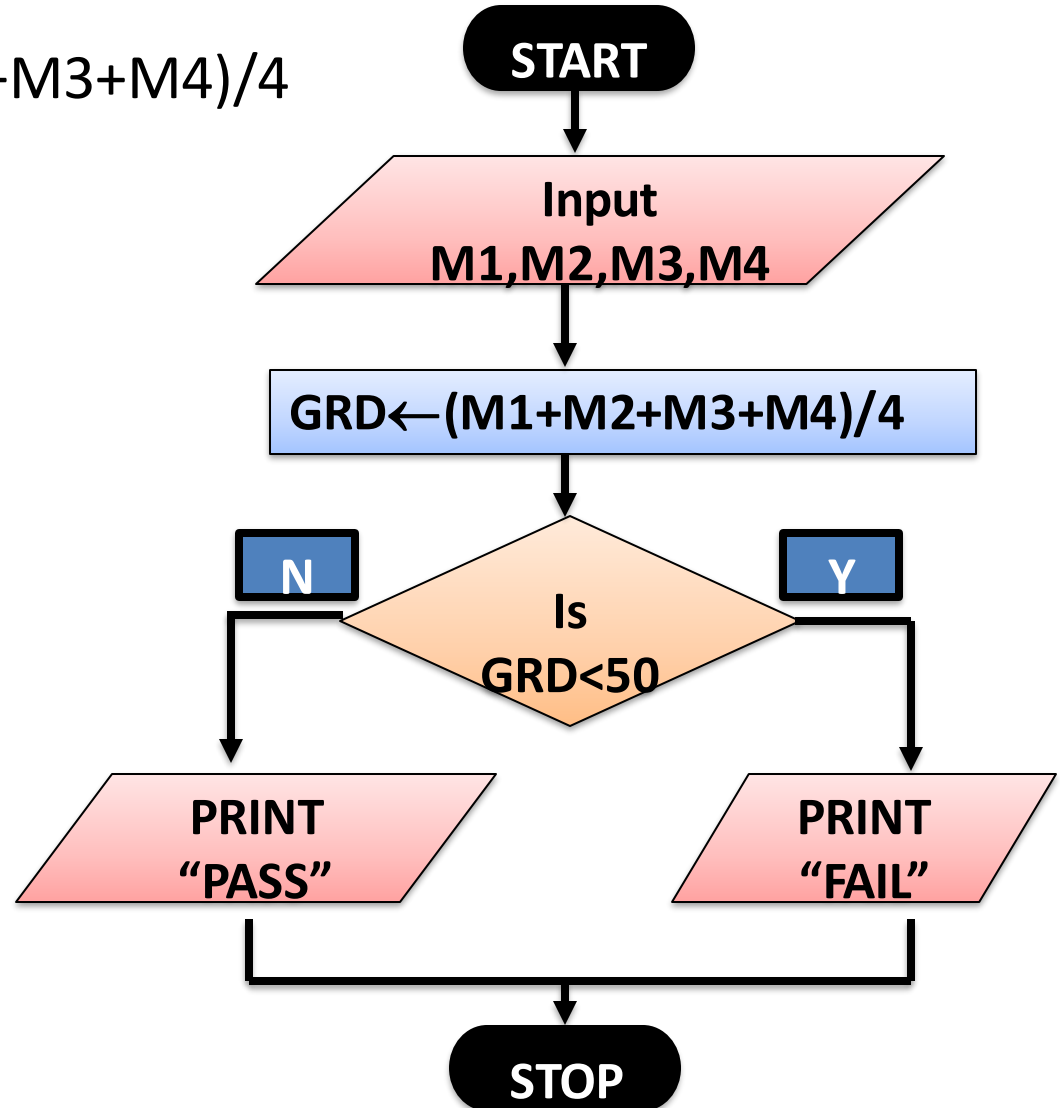


Decision Structure



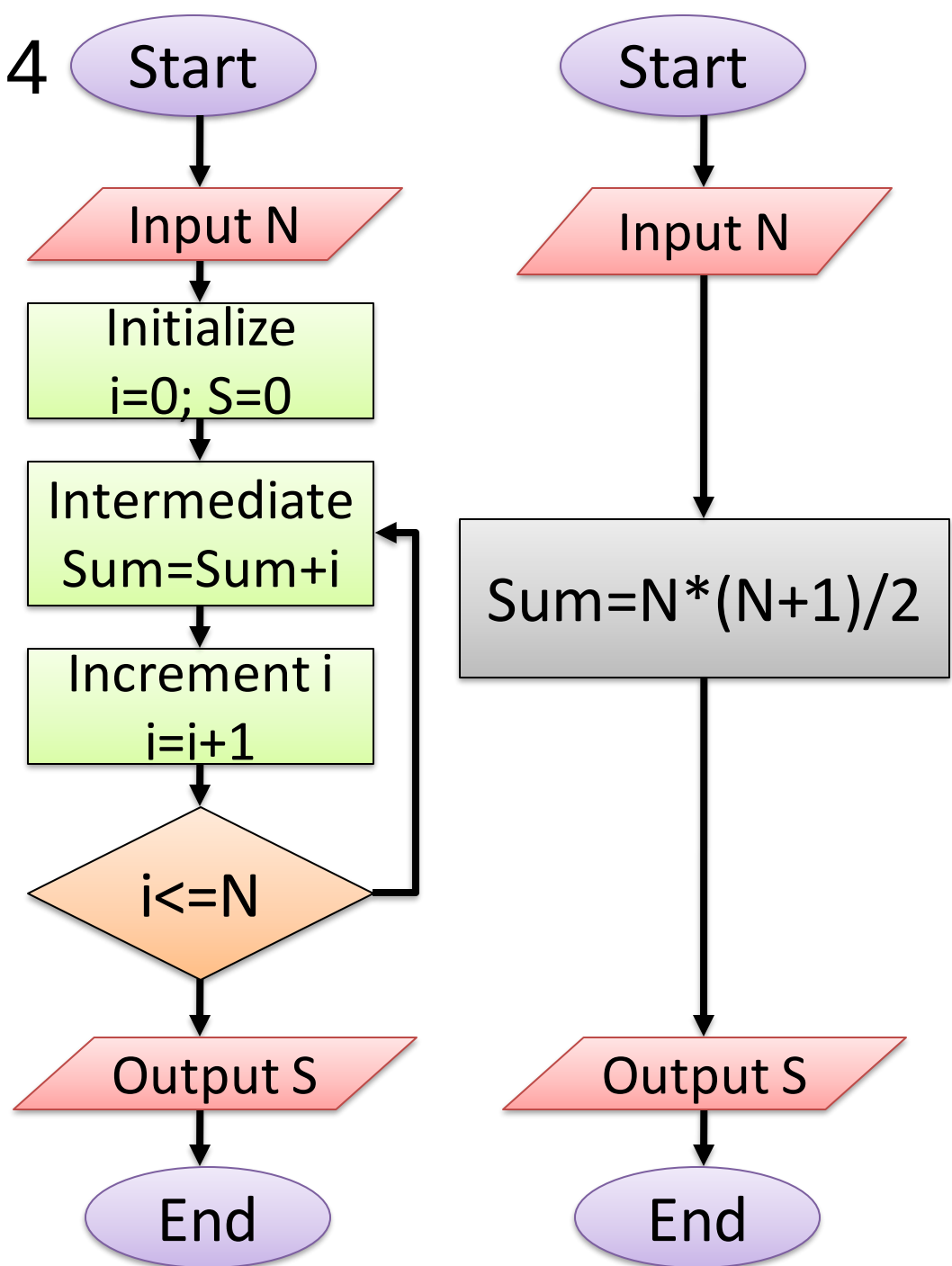
Example 4 : Grade Calculation

- Step 1: Input M1,M2,M3,M4
- Step 2: $GRD \leftarrow (M1+M2+M3+M4)/4$
- Step 3: if (GRD < 50) then
 Print "FAIL"
else
 Print "PASS"
endif



Flow Chart Example 4

- Sum of first N natural number
- Two methods
 - Method 1
 $\text{Sum} = 1 + 2 + 3 + \dots + N$
 - Method 2
 $\text{Sum} = N(N+1)/2$



C Programming : Sum of A and B

```
#include <stdio.h>
```

```
int main() {
```

```
    int A, B, S;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d", &A, &B);
```

```
    S=A+B;
```

```
    printf("Res=%d", S);
```

```
    return 0;
```

```
}
```

Header file:
Standard Input/Output

Variable Declarations

Printing message

Asking for inputs

Compute

Output Result

Identifier in C : Name

- Is a unique name
 - That simply references to memory locations, which can hold values (data).
- Identifiers give unique names
 - To various objects in a program
 - To **variable and function**
- Are formed by
 - Combining letters (both upper and lowercase)
 - Digits (0–9)
 - Underscore (_).

Rules for Naming Identifier in C

- First character of an identifier
 - Must be a letter (non-digit) including underscore (_).
- Space: Blank or white space character is not permitted
Space, Tab
 - linefeed, carriage-return, form-feed, vertical-tab, and newline characters
- Length
 - Can be any length but implementation dependent
- **Reserved words/keywords cannot be used.**

Identifier in C : Variable Name Examples

Correct	Wrong
secondName	2ndName /* starts with a digit */
_addNumber	%calculateSum /* contains invalid character */
charAndNum	char /* reserved word */
annual_rate	annual rate /* contains a space */
stage4mark	my\nName /* contains new line character, \n */

Variables in C

- **Variables** in C have the same meaning as variables in algebra.
- They represent some unknown, or variable, value.

$$x = a + b$$

$$z + 2 = 3(y - 5)$$

- Generally variables in algebra are represented by a single alphabetic character.

Variables in C

- Variables are
 - Named blocks of memory
 - Valid identifier.
- Variable have two properties in syntax:
 - Name — a unique identifier
 - Type — what kind of value is stored.
- Every variable stored in the computer's memory
 - Has a name, a value and a type.
 - Value may change during the program execution.

C Variable : Example

Correct

```
int    x, y, z, my_data = 4;
```

```
short  number_one;
```

```
long   TypeofCar;
```

```
unsigned int  positive_number;
```

```
char    Title;
```

```
float    commission, yield = 4.52;
```

```
char the_initial = 'M'; //A char
```

```
char studentName[20]="India";//A string
```

C Variable : Example

Wrong	Comments
<code>int 3a, 1, -p;</code>	Start with digit or -
<code>short number+one;</code>	Have +
<code>long #number;</code>	Start wit #

Variable Naming Conventions

- Standard Conventions
 - Begin variable names with lowercase letters
 - Use meaningful identifiers
 - Separate “words” within identifiers with underscores or mixed upper and lower case.
 - Examples: `surfaceArea`, `surface_Area`, `surface_area`
 - **Be consistent!**

Case Sensitivity

- Each alphabet used is **case sensitive**
 - It matters whether an **identifier**, such as a variable name, is uppercase or lowercase.
 - Example:

area

Area

AREA

ArEa

are all seen as **different** variables by the compiler.

Which Are Legal Identifiers?

AREA	area_under_the_curve
3D	num45
Last-Chance	#values
x_yt3	pi
num\$	%done
lucky***	

Which Are Legal Identifiers?

AREA

3D 

Last-Chance 

x_yt3

num\$ 

lucky*** 

area_under_the_curve

num45

#values 

pi

%done 

Naming Conventions

- Use all uppercase for **symbolic constants** (used in **#define** preprocessor directives).
- Examples:

```
#define PI 3.14159
```

```
#define AGE 52
```

C Programming : Sum of A and B

```
#include <stdio.h>
```

```
int main() {
```

```
    int A, B, S;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d", &A, &B);
```

```
    S=A+B;
```

```
    printf("Res=%d", S);
```

```
    return 0;
```

```
}
```

Header file:
Standard Input/Output

Variable Declarations

Printing message

Asking for inputs

Compute

Output Result

C Keywords/Reserve Words

- Reserved words are : not available for redefinition
- Can not be used as variable name
- Have special meaning in C

auto	extern	sizeof	for
break	float	static	goto
case	inline	struct	if
char	int	switch	else
const	long	typedef	enum
continue	register	union	do
default	restrict	unsigned	void
double	return	volatile	short
		while	signed

Data type and Size

- In C, data type categorized as:
 - Primitive Types in ANSI C
 - `char`, `short`, `int`, `float` and `double`.
 - `long int`, `long double`
 - User Defined Types
 - `struct`, `union`, `enum` and `typedef`
(will be discussed later).
 - Derived Types
 - `pointer`, `array` and `function pointer`
(will be discussed later).

Numeric Data Type

- **char, short, int, long int**
 - char : 8 bit number (1 byte=1B)
 - short: 16 bit number (2 byte)
 - int : 32 bit number (4B)
 - long int : 64 bit number (8B)
- **float, double, long double**
 - float : 32 bit number (4B)
 - double : 64 bit number (8B)
 - long double : 128 bit number (16B)

Numeric Data Type

- **char, short, int, long int**
 - We have : Signed and unsigned version
 - char (8 bit), int (32 bit)
 - char : -128 to 127
 - unsigned char: 0 to 255
 - int : -2^{31} to $2^{31}-1$
 - unsigned int : 0 to $2^{32}-1$
- **float, double, long double**
 - For fractional, real number data
 - All these numbered are signed and get stored in different format

Numeric Data Type



unsigned char



char



unsigned short



short



Unsigned int

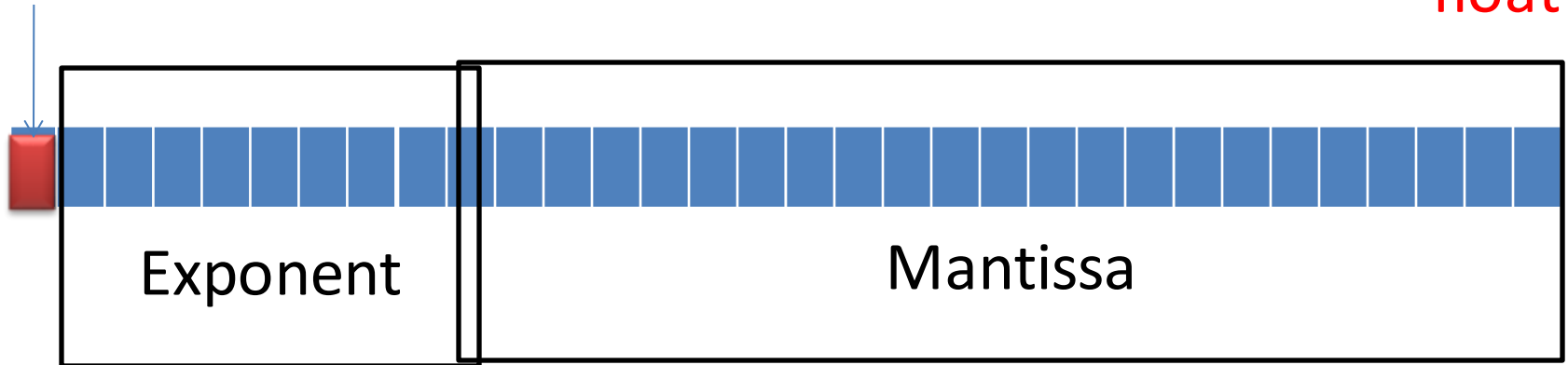


int

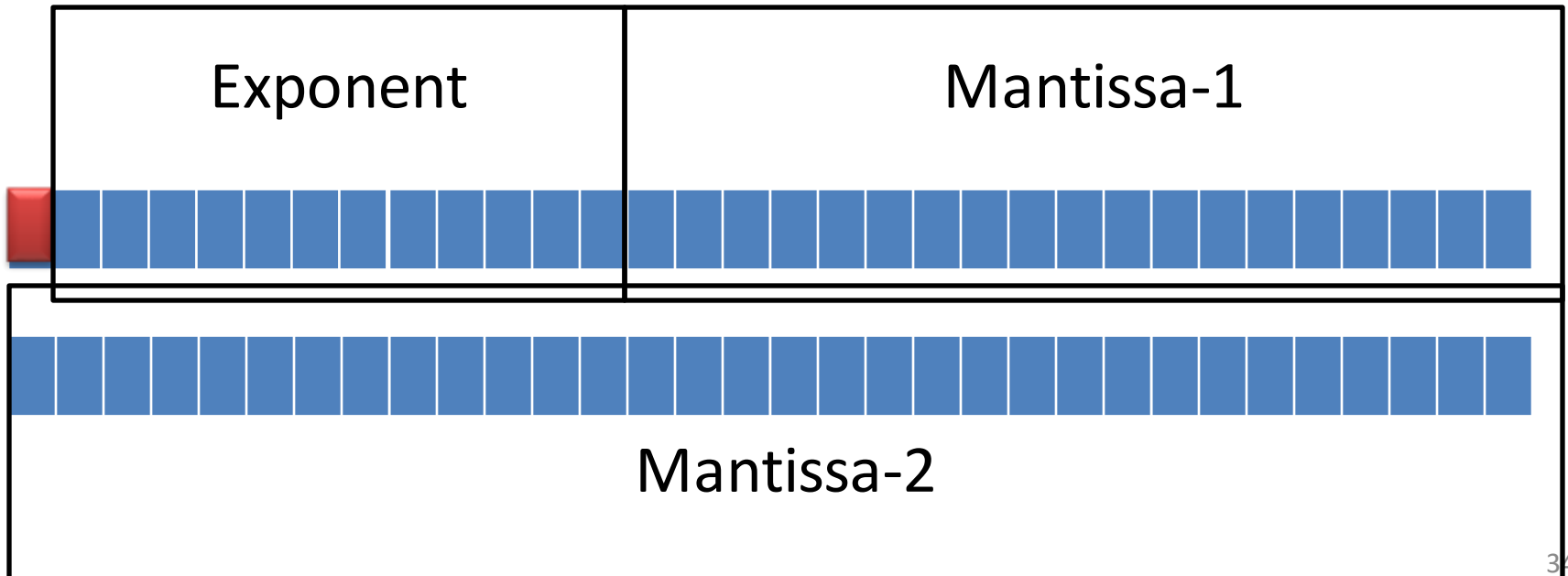
Numeric Data Type

Sign bit

float



double



Testing size of Numeric Data

```
#include<stdio.h>
int main(){
    printf("size of char %d\n", sizeof(char)); //1
    printf("size of short %d\n", sizeof(short)); //2
    printf("size of int %d\n", sizeof(int)); //4
    printf("size of long int %d\n", sizeof(long int)); //8
    printf("size of float \n", sizeof(float)); //4
    printf("size of double %d\n", sizeof(double)); //8
    printf("size of long double %d\n", sizeof(long double)); //16
    return 0;
}
```

Thanks